

Assignment 1: Designing Interacting Objects - Uno

1. Introduction

For this assignment, our objective was to practice designing software from an object-oriented perspective. This meant we picked a scenario that interested us and organized it into classes. Then, identified the relationships between the classes and decided which behaviors should be shared or overridden; and finally, developed interactions between the objects.

I decided to create an object-oriented implementation of the card game Uno where a coordinator class drives automated players through full games using the official rules.

2. The Card Class Hierarchy

The first file implemented was “*card.py*” which defines the `Card` class hierarchy. The classes found within this file are:

- `Card`, `NumberCard`, `PlusTwoCard`, `SkipCard`, `ReverseCard`, `WildCard` and `WildDrawFourCard`.

This parent class within this file is class `Card` which holds all of the shared data and default behavior. The methods include:

- `__init__(self, color)`
 - This method initializes the card color to Red, Yellow, Green, Blue, or Wild.
- `matches(self, other_card, current_color)`
 - This method defines the default matching rule where cards of the same color count as a match.
- `play(self, game)`
 - This method applies a card’s effect to the game state. The parent class `Card` will place the card in the discard pile, and the child classes will override this to add real behavior such as skip, reverse, draw, wild, or wild and draw.
- `__str__(self)`
 - This method returns a string representation of a `Card` object.

The first child class is `NumberCard` which represents a standard number card in Uno (0-9).

The methods include:

- `__init__(self, color, number)`
 - This method inherits the color initialization from the parent class `Card` and extends the method to initialize a card’s number, if applicable.
- `matches(self, other_card, current_color)`

- This method overrides the default matches method from the parent class `Card` to check if the card matches both the number and color of the previously played card.
- `play(self, game)`
 - This method inherits the same behavior from the parent class `Card` and simply places a card in the discard pile.
- `__str__(self)`
 - This method overrides the default method from the parent class `Card` to return a string representation of a `NumberCard` object.

The child class `PlusTwoCard` represents an action card in Uno which forces the next player to draw two cards and lose their turn. The methods include:

- `__init__(self, color)`
 - This method inherits the color initialization from the parent class `Card`.
- `play(self, game)`
 - This method inherits the behavior from the default play method of parent class `Card` and extends it by forcing the next player to draw two cards and be skipped.
- `__str__(self)`
 - This method overrides the default method from the parent class `Card` to return a string representation of a `PlusTwoCard` object.

The child class `SkipCard` represents an action card in Uno which skips the next player's turn. The methods include:

- `__init__(self, color)`
 - This method inherits the color initialization from the parent class class `Card`.
- `play(self, game)`
 - This method inherits the behavior from the default play method of parent class `Card` and extends it by skipping the next player's turn.
- `__str__(self)`
 - This method overrides the default method from the parent class `Card` to return a string representation of a `SkipCard` object.

The child class `ReverseCard` represents an action card in Uno which reverses the order of play. The methods include:

- `__init__(self, color)`
 - This method inherits the color initialization from the parent class `Card`.
- `play(self, game)`
 - This method inherits the behavior from the default play method of parent class `Card` and extends it by reversing the order of play.
- `__str__(self)`
 - This method overrides the default method from the parent class `Card` to return a string representation of a `ReverseCard` object.

The child class `WildCard` represents a wild card in Uno which changes the current color to one of the player's choice. The methods include:

- `__init__(self)`
 - This method inherits the color initialization from the parent class `Card` and sets it to "Wild".
- `matches(self, other_card, current_color)`
 - This method overrides the default matches method from parent class `Card` to allow wild cards to always be played, since they can be played no matter what the previous card is.
- `play(self, game)`
 - This method inherits the behavior from the default play method of parent class `Card` and extends it by changing the current color to one of the player's choice.
- `__str__(self)`
 - This method overrides the default method from the parent class `Card` to return a string representation of a `WildCard` object.

The child class `WildDrawFourCard` inherits from the parent class `WildCard` and represents a wild draw four card in Uno, which changes the current color to one of the player's choice and forces the next player to draw four cards. The methods include:

- `__init__(self)`
 - This method inherits the color initialization from class `WildCard`.
- `matches(self, other_card, current_color)`
 -
- `play(self, game)`
 - This method inherits the behavior from the play method of parent class `WildCard` and extends it by also forcing the next player to draw four cards.
- `__str__(self)`
 - This method overrides the default method from the parent class `WildCard` to return a string representation of a `WildDrawFourCard` object.

3. The Player Class

The next file implemented was "`player.py`" which defines the `Player` class. A `Player` object holds a hand and only acts when the `Game` coordinator calls its methods, so it doesn't act on its own. This file also imports the `random` library and the `WildCard` child class.

The only class within this file is class `Player` which holds each player's card hand. The methods include:

- `__init__(self, name)`
 - This method initializes a player's name and their respective card hands.
- `draw(self, deck)`
 - This method allows the player to draw a card from the deck.

- `has_playable_card(self, current_card, current_color)`
 - This method checks if a player has a playable card in their hand.
- `choose_card(self, current_card, current_color)`
 - This method picks the first legal card found in a player's hand. The color will be checked first and if there are no matches, then it will check if the player has a wild.
- `choose_color(self)`
 - This method picks the color a player has the most of.
- `__str__(self)`
 - This method returns a string representation of a `Player` object (i.e. a player's name).

4. The Game class

The next file implemented was `"game.py"` which defines the `Game` class. This is what drives interaction between `Card` and `Player` objects. This file imports the `random` and `time` libraries in addition to the child classes from class `Card` (`NumberCard`, `PlusTwoCard`, `SkipCard`, `ReverseCard`, `WildCard`, `WildDrawFourCard`) and the `Player` class. The methods include:

- `__init__(self, player_name)`
 - This method initializes the game state, specifically the: player's names, card deck, discard pile, game direction, player turn index, and current color of the top card.
 - Additionally, the methods `_build_deck()`, `_deal_starting_hands()`, and `_flip_starting_card` are called during initialization.
- `_build_deck(self)`
 - This method builds a deck of 108 cards and shuffles it. The single leading underscore indicates that the name is meant for internal use only.
 - 76 numbered cards
 - 19 cards of each color
 - 1 number zero card and two sets of cards numbered 1-9
 - 24 action cards (skip, reverse, and plus two)
 - 2 of each action card
 - 6 total of each color
 - 8 wild cards
 - 4 wild cards and 4 wild draw four cards
- `_deal_starting_hands(self, hand_size=7)`
 - This method deals all players 7 cards each from the deck. 7 is hardcoded into the parameters because the official Uno rules indicate that each player

receives 7 cards. The single leading underscore indicates that the name is meant for internal use only.

- `_flip_starting_card(self)`
 - This method flips the top card from the draw pile and places it in the discard pile, to start the game. The single leading underscore indicates that the name is meant for internal use only.
- `current_player(self)`
 - This method returns the name of a player in the game.
- `advance_turn(self)`
 - After a card is placed, this method moves the game to the next player's turn.
- `top_card(self)`
 - This method returns the most recently played card.
- `reshuffle_if_needed(self)`
 - This method creates a new draw pile by reshuffling the discard pile and uses the last played card as the new top card of the discard pile.
- `play_turn(self)`
 - This method runs through the turn of a single player
 - If the player does not have a playable card,
 - They will draw a card from the discard pile and have their turn skipped.
 - If the player does have a playable card
 - They will choose a playable card from their hand and place it
 - If the card is wild, the current color will be updated to reflect the player's choice.
 - If the card is not wild, the current color will remain the same.
 - If the player has one card left, a message will be output notifying players that the player has one card remaining
 - This is reminiscent of the official Uno rules in which players must vocalize they have one card left by saying "Uno!".
 - If the player has no more cards left, a message will be output notifying players who won and the game will end
- `run(self)`
 - This method starts the game.

5. The Main File

The last file implemented was "*main.py*" which is the main entry point of the game. This file imports the `Game` class, then a variable `players` is declared that defines the list of players. Finally, a new game is created using the `players` variable and the game is started.

6. Challenges

One of the challenges I faced during this assignment was figuring out all the different functions I needed, in order for my program to run intuitively. Uno has a lot of rules and details that don't really come to mind when you're physically playing it in real life. Most players aren't thinking about how many cards are found in a singular deck, or how many of each card type is found in a deck; but, these are things you have to think about when you're coding the game. If the cards aren't coded correctly, then will there be an unlimited number of action cards? Will the game ever end? Small details like these made it challenging to organize the class methods.

Another challenge I faced was the games never ending. The number of cards each player had kept rising and I couldn't understand why that was happening. After debugging my code for some time, I realized that there was a parameter mismatch across my `matches()` method in my `card.py` file. In `player.py`, every call to `matches()` uses this order:

- `c.matches(current_card, current_color)`

However, parent class `Card` and child class `NumberCard` define it as:

- `def matches(self, current_color, other_card)`

The parameter names are swapped, so whenever python runs

`c.matches(current_card, current_color)`, the actual `current_card` gets assigned to the parameter named `current_color`, and the actual color string gets assigned to the parameter named `other_card`. Inside the method:

- `return self.color == current_color`
 - Comparing a string to a `Card` object is never `True`
- `If isinstance((other_card, NumberCard) and self.number == other_card.number:`
 - `other_card` is actually a string here, never a `NumberCard` object

Due to this, these checks fail every single time for every `NumberCard`, `SkipCard`, `PlusTwoCard`, and `ReverseCard` since they all inherit the broken version from parent class `Card`. On the other hand, `WildCard.matches()` had the parameters in the correct order (`other_card, current_color`) but it ignores them completely and just returns `True`. This meant wild cards are the only ones that registered as playable, which is why I was seeing: player's hand sizes balloon, No regular cards being played, and only wild/wild draw four cards being played. This goes hand-in-hand with the challenge I mentioned before.

7. Conclusion

This assignment was a much needed re-introduction back into the world of object-oriented programming. The last class I took that revolved around OOP specifically was the introductory class back in 2023, so getting to take another class that specifically revolves around OOP is so helpful. I really enjoyed orienting my design around Uno because it's a game I've played since I was child, so being able to break it down and code it was very interesting to say the least.

8. Resources

Uno. "Original Uno Rules." *Original Uno Rules*, 2009, <https://www.unorules.com>. Accessed 14 September 2026.

Uno. "UNO Cards—UNO!™ – the Official UNO mobile game." *UNO!*, 29 12 2017, https://www.letsplayuno.com/news/guide/20181213/30092_732567.html. Accessed 14 September 2026.